

Aula 9 - Título da Aula

Professor: André L. Marcato

Universidade Federal de Juiz de Fora
Engenharia Elétrica — Robótica e Automação Industrial

January 30, 2026

Roteiro

- ① Sistema de Launch
- ② Funcionalidades Avançadas
- ③ Prática

Roteiro

① Sistema de Launch

② Funcionalidades Avançadas

③ Prática

ROS 2 Launch System

- **Problema:** Iniciar 10 nós manualmente em 10 terminais é inviável.
- **Launch:** Sistema para automatizar a execução e configuração de nós.
- **Diferença do ROS 1:**
 - ROS 1 usava XML (`.launch`).
 - ROS 2 usa **Python** (`.launch.py`). Isso permite lógica complexa (if/else, loops, busca de diretórios dinâmicos).
- **Conceito:** Você descreve o estado final desejado do sistema.

Estrutura Básica (.launch.py)

- Deve retornar um objeto LaunchDescription contendo uma lista de **Actions** (Nós, Processos, Inclusões).

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            executable='draw_square',
            name='draw'
        )
    ])
```

Argumentos e Dinâmica

- **DeclareLaunchArgument**: Define variáveis de entrada (ex: cor, velocidade, namespace) com valor default.
- **LaunchConfiguration**: Captura o valor desse argumento durante a execução.

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    # 1. Declara o argumento 'bg_color'
    bg_color_arg = DeclareLaunchArgument(
        'bg_color', default_value='255', description='Cor de fundo'
    )

    # 2. Usa o valor no parametro do n
    node = Node(
        package='turtlesim',
        executable='turtlesim_node',
        parameters=[{'background_r': LaunchConfiguration('bg_color')}]
```

Roteiro

① Sistema de Launch

② Funcionalidades Avançadas

③ Prática

Params, Remaps e Namespaces

- Podemos injetar configurações diretamente na instanciação do Node.

```
Node(  
  package='my_package',  
  executable='my_node',  
  name='custom_name',      # Renomeia o n  
  namespace='robot1',     # Agrupa em /robot1/...  
  parameters=[  
    {'param_int': 10},    # Dicionrio direto  
    'config/params.yaml' # Ou caminho para arquivo  
  ],  
  remappings=[  
    ('/old_topic', '/new_topic'),  
    ('/cmd_vel', '/robot1/cmd_vel')  
  ]  
)
```

Inclusão e Modularidade

- Launchs podem chamar outros Launchs (IncludeLaunchDescription).
- Útil para separar a lógica (ex: um launch para sensores, outro para navegação, um "main" que chama todos).

```
import os
from ament_index_python.packages import get_package_share_directory
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import
    PythonLaunchDescriptionSource

# ... dentro da lista do LaunchDescription:
IncludeLaunchDescription(
    PythonLaunchDescriptionSource([
        os.path.join(get_package_share_directory('outro_pkg'), 'launch'),
        '/sensor_launch.py'
    ]),
    launch_arguments={'sensor_id': '1'}.items(),
)
```

Roteiro

- ① Sistema de Launch
- ② Funcionalidades Avançadas
- ③ Prática

Prática — Ajuste do setup.py

Importante

Arquivos de launch não são código-fonte compilado, eles precisam ser copiados para a pasta share na instalação.

```
# setup.py
import os
from glob import glob
# ...
data_files=[
    ('share/ament_index/resource_index/packages', ['resource/' +
    package_name]),
    ('share/' + package_name, ['package.xml']),

    # ADICIONE ESTA LINHA:
    (os.path.join('share', package_name, 'launch'), glob('launch/*.launch
    .py')),
],
```

- Crie a pasta launch/ na raiz do pacote (ao lado de src,

Prática — my_app.launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # N 1: Publisher (com remap)
        Node(
            package='example_interfaces',
            executable='one_off_timer_control', # Apenas exemplo
            remappings=[('/one_off_timer_control', '/cmd_start')]
        ),
        # N 2: Turtlesim (com namespace)
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            namespace='turtles',
            parameters=[{'background_b': 200}]
        )
    ])
]
```

Prática — Compilar e Executar

- Sempre compile após criar/modificar launch files (para que sejam copiados para install/share).

```
$ colcon build --symlink-install  
$ source install/setup.bash  
  
# Sintaxe: ros2 launch <pacote> <arquivo>  
$ ros2 launch my_pkg my_app.launch.py
```

- Passando argumentos via CLI:

```
$ ros2 launch my_pkg my_app.launch.py bg_color:=100
```

Exercício Final

Orquestrador de Tartarugas

Crie um arquivo `two_turtles.launch.py` que:

- 1 Inicie **duas janelas** do Turtlesim.
- 2 Use **Namespaces** diferentes para não colidirem (ex: `t1` e `t2`).
- 3 Declare um **Launch Argument** chamado `speed` (embora o turtlesim não use `speed` diretamente, use-o para imprimir um `LogInfo` ou passar como parâmetro fictício para treinar).
- 4 Remapeie o tópico `/t1/turtle1/cmd_vel` para `/control_signal`.